

Análise Dinâmica: Depuração

Técnicas de Evasão de Malware

As técnicas de evasão são estratégias projetadas para impedir que ferramentas de segurança e analistas compreendam o real propósito de um arquivo malicioso.

1. Evasão de Análise Estática

O objetivo aqui é enganar quem está "lendo" o código sem executá-lo.

- **Alteração de Hash:** Mudar um único bit (adicionando uma instrução NOP - No Operation) gera um hash (MD5/SHA256) completamente diferente, invalidando listas de bloqueio baseadas em assinaturas de hash.
- **Ofuscação de Strings:** URLs de C2 (Comando e Controle) e comandos críticos são criptografados ou codificados (ex: Base64). Eles só aparecem de forma legível na memória durante a execução.
- **Packing (Empacotamento):** O malware é compactado ou criptografado dentro de um "stub". Para o analista, o arquivo parece um amontoado de dados sem sentido até que o stub o descompacte na memória.
- **Dynamic Loading:** Em vez de listar todas as funções que vai usar no cabeçalho do arquivo (IAT - Import Address Table), o malware usa LoadLibrary para chamar DLLs apenas quando necessário, escondendo suas intenções reais.

2. Evasão de Análise Dinâmica

O objetivo aqui é detectar se o ambiente de execução é uma "armadilha" (Sandbox/VM).

A. Detecção de Virtualização (Anti-VM)

O malware procura por "rastros" de softwares como VirtualBox ou VMware:

- **Arquivos/Drivers:** Procura por VBoxGuest.sys ou vmtoolsd.exe.
- **Registro:** Verifica chaves como
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Enum\PCI\VEN_80EE .
. . (ID do VirtualBox).
- **Hardware Limitado:** Se a máquina tiver apenas 1 CPU, 2GB de RAM e 40GB de disco, o malware assume que é uma VM de análise e não executa a carga maliciosa.

B. Ataques de Temporização (Timing Attacks)

- **Sleep Skipping:** O malware executa um Sleep de 24 horas. Sandboxes automatizadas tentam "pular" esse tempo. O malware então verifica o relógio do sistema: se o tempo "passou" rápido demais, ele sabe que está sendo manipulado.

C. Verificação de Atividade Humana

Uma Sandbox recém-criada é "limpa" demais. O malware verifica:

- Movimentação do mouse e cliques.
- Existência de arquivos recentes (Word, PDFs).
- Histórico de navegação no browser.
- Se não houver interação humana, o malware permanece benigno.

D. Detecção de Ferramentas (Anti-Analysis)

O malware varre a lista de processos (Process32First/Next) em busca de:

- `procmon.exe`, `procexp.exe`, `wireshark.exe`, `x64dbg.exe`.
- Se encontrar qualquer um desses, ele se fecha imediatamente ou executa uma função inofensiva (como abrir a calculadora).

Depuração Interativa (Debugging)

A depuração é o processo de executar o malware de forma controlada, instrução por instrução, para observar mudanças na memória, nos registradores da CPU e nas variáveis. Para um analista, o debugger é a ferramenta que permite **burlar técnicas de evasão** (como Anti-VM e Anti-Debug) "mentindo" para o malware em tempo real.

1. Por que depurar um malware?

- **Controle de Fluxo:** Você pode alterar o valor de um registrador (ex: mudar EAX de 0 para 1) para forçar o malware a seguir um caminho que ele normalmente evitaria.
- **Extração de Payload:** Se o malware estiver "empacotado" (packed), você pode deixá-lo se descompactar na memória, pausar a execução e fazer o *dump* do código limpo.
- **Análise de Ofuscação:** Ver as strings sendo decodificadas na RAM logo antes de serem usadas.

2. Categorias de Depuradores

Nível	Tipo	Uso Principal	Exemplo de Ferramenta

Alto	Código-fonte	Usado por desenvolvedores que possuem o arquivo .c ou .py. Foca em nomes de variáveis reais.	Visual Studio Debugger, PyCharm
Médio	Assembly	O padrão para Malware Analysis. Trabalha com binários compilados (.exe, .dll). Foca em registradores (EAX, ESP) e endereços de memória.	x64dbg , OllyDbg, Immunity Debugger
Baixo	Kernel	Usado para analisar drivers maliciosos ou Rootkits que operam no "coração" do Windows (Ring 0).	WinDbg , KD

3. Conceitos Cruciais para a Próxima Tarefa

Para operar um debugger de nível de Assembly, você precisará dominar:

- **Breakpoints (Pontos de Interrupção):** Comandos que dizem ao debugger: "Pare a execução exatamente aqui".
- **Stepping (Passo a passo):**
 - *Step Into:* Entra dentro de uma função.
 - *Step Over:* Executa a função inteira e para na próxima linha (útil para ignorar funções conhecidas do Windows).
- **Registradores:** Pequenas áreas de armazenamento ultra-rápido na CPU. O malware usa EAX frequentemente para armazenar resultados de funções (0 geralmente significa falha, 1 sucesso).

Dominando o Depurador (x64dbg / x32dbg)

O x64dbg é um depurador de nível de Assembly de código aberto para Windows. Ele é dividido em quatro painéis principais na aba **CPU**, que trabalham juntos para mostrar o estado exato do malware:

1. Os Quatro Painéis da CPU

- **Desmontagem (Superior Esquerdo):** Mostra o código do malware em Assembly. A linha em destaque é a próxima instrução que a CPU vai executar (apontada pelo registrador EIP ou RIP).
- **Registradores (Superior Direito):** Exibe o conteúdo atual das "pastas de trabalho" da CPU (EAX, EBX, ECX, etc.).
 - *Dica de Analista:* Observe o EAX após uma função de verificação; ele geralmente contém o resultado (0 para falso, 1 para verdadeiro).
- **Dump (Inferior Esquerdo):** Permite visualizar o conteúdo bruto da memória RAM em Hexadecimal ou Strings. É aqui que você vê dados descryptografados.

- **Stack / Pilha (Inferior Direito):** Mostra os dados temporários, endereços de retorno e argumentos de funções que o programa está usando no momento.

Abas de Navegação Críticas

Além da visualização da CPU, o depurador oferece abas que organizam os recursos do sistema:

- **Breakpoints (Pontos de Interrupção):** Lista onde você ordenou que o programa pare. Sem breakpoints, o malware executaria até o fim antes que você pudesse ver qualquer coisa.
- **Memory Map (Mapa de Memória):** Mostra todas as seções do programa (.text, .data, .rsrc) e bibliotecas (DLLs) carregadas. Útil para encontrar código injetado ou seções com permissão de execução (E).
- **Call Stack:** Mostra a "árvore genealogia" das funções. Se o malware travar ou chamar uma API suspeita, aqui você vê quem chamou quem até chegar naquele ponto.
- **Handles:** Lista arquivos, chaves de registro e objetos de rede que o malware mantém abertos.

Análise de Evasão: TLS Callbacks e Anti-Debugging

Ao depurar o crackme-arebel.exe, identificamos uma rotina de proteção que tenta congelar o analista. Abaixo estão os pontos chave para a documentação:

1. Qual chamada de API no exemplo é usada para enumerar os processos em execução e buscar o depurador?

- **Análise:** O malware tira um "snapshot" de todos os processos ativos no Windows para comparar os nomes com uma lista de ferramentas de análise (como x32dbg.exe).
- **Resposta:** CreateToolhelp32Snapshot

2. Qual é o valor da Zero Flag (ZF) no momento do salto condicional (JZ/JNZ) logo no início do TLS Callback?

- **Análise:** As Flags da CPU determinam se um salto será tomado ou não. No x32dbg, observamos que ZF estava em 1, indicando que a condição para o salto de "segurança" (evasão) não foi atendida.

- Resposta: 1

```

ESP 006FF7E4 &"<æ[_^]Ã\x10
ESI 006FF7F4
EDI 002A1160 crackme-arebe

EIP 002A1160 crackme-arebe

EFLAGS 00000244
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 000036B7 (ERROR_SXS_K
LastStatus C0150008 (STATUS_SXS_

```

3. O salto condicional mencionado acima foi executado automaticamente pelo programa? (S/N)

- **Análise:** Devido ao estado dos registradores no ambiente de análise, o programa seguiu o caminho "malicioso" que leva à suspensão da thread, ignorando o salto que o levaria para a execução normal.
- Resposta: N

ECX EDI	002A115D	CC	int3
	002A115E	CC	int3
	002A115F	CC	int3
	002A1160	55	push ebp
	002A1161	8BEC	mov ebp,esp
	002A1163	837D 0C 01	cmp dword ptr ss:[ebp+C],1
	002A1167	75 05	jne crackme-arebel.2A116E
EIP	002A1169	E8 92FEFFFF	call crackme-arebel.2A1000
	002A116E	5D	pop ebp
	002A116F	C2 0C00	ret c
	002A1172	CC	int3
	002A1173	CC	int3
	002A1174	CC	int3
	002A1175	CC	int3
	002A1176	CC	int3
	002A1177	CC	int3
	002A1178	CC	int3
	002A1179	CC	int3
	002A117A	CC	int3
	002A117B	CC	int3

4. De qual DLL do Windows a API SuspendThread (responsável por travar o depurador) está sendo carregada?

- **Análise:** Identificar a biblioteca de origem ajuda a mapear como o malware interage com o kernel para manipular threads do sistema.
- **Resposta:** KERNEL32.dll

6B: 'k'	53: 'S'
65: 'e'	75: 'u'
72: 'r'	73: 's'
6E: 'n'	70: 'p'
65: 'e'	65: 'e'
6C: 'l'	6E: 'n'
33: '3'	64: 'd'
32: '2'	54: 'T'
2E: '.'	68: 'h'
64: 'd'	72: 'r'
6C: 'l'	65: 'e'
6C: 'l'	61: 'a'
	64: 'd'